Eval đầy đủ với Docker: Claude Code headless (claude-opus-5-5, không preset) — 15/10/2026

Tiếp nối EVAL_CLAUDECODE_2026-10-14.md. Hôm 14/10 chỉ chạy được cấu hình `llm` vì Docker tắt. Lần này Docker bật, chạy đủ ba cấu hình trên bộ Python và hai cấu hình trên bộ đa ngôn ngữ, với cấu hình server đã được ablation hôm trước chọn là tốt nhất: **`SYSTEM_MODE=custom`** (chỉ SKILL.md, không preset Claude Code), `CLAUDE_EFFORT=medium`, `CONCURRENCY=2`, `REQUEST_TIMEOUT_MS=300000`.

Lệnh:

```
# cửa sổ 1
cd claude-server; $env:SYSTEM_MODE="custom"; npm start
# cửa sổ 2 (static, both nối vào thư mục llm đã có từ 14/10; eval tự bỏ qua case đã xong)
python eval/run_eval.py --configs static,both --profile claudecode --out eval/out-claudecode-custom
python eval/run_eval.py --truth eval/ground_truth_multi.json --configs llm,both --profile claudecode --out eval
python eval/score.py --out eval/out-claudecode-custom            # + --tolerance 0
python eval/score.py --truth eval/ground_truth_multi.json --out eval/out-multi-claudecode
```

Cùng prompt, luật, fact-check, cách ghim và cách chấm với các lần đo Gemini (`eval/out-v3-rescored`, `eval/out-multi-v3-rescored`) nên so trực tiếp được. Static chạy trong sandbox Docker `hermesqa-runner` (semgrep, bandit, gitleaks, hadolint, ruff), `--network none`.

1. Bộ Python (35 case)

Config	TP	FP	FN	Precision	Recall	F1	s/PR (TB)	s/PR (trung vị)	tokens/PR
`static`	21	5	15	80,8%	58,3%	67,7%	106,9	115,7	0
`llm` (14/10)	35	17	1	67,3%	97,2%	79,5%	35,5	36,5	14 163
`both`	33	13	3	71,7%	91,7%	**80,5%**	176,5	167,9	12 837

Tham chiếu Gemini 3.1 flash-lite (`eval/out-v3-rescored`, 07/10): `static` 70,0 / 58,3 / 63,6 · `llm` 85,0 / 94,4 / 89,5 · `both` 81,0 / 94,4 / 87,2.

Theo category (bắt được / tổng):

Category	static	llm	both
bug	4/7	7/7	7/7
devops	4/8	8/8	**6/8**
performance	0/1	1/1	1/1
security	12/13	12/13	12/13
style	1/3	3/3	3/3
test	0/4	4/4	4/4

Theo vai / công cụ (finding / TP / FP / precision):

Nguồn	`llm`	`both`
SE	20 / 17 / 3 / 85%	7 / 5 / 2 / 71%
QA	23 / 10 / 13 / 44%	11 / 5 / 6 / 46%
DevOps	9 / 8 / 1 / 89%	2 / 2 / 0 / 100%
semgrep	–	14 / 11 / 3 / 79%
bandit	–	5 / 3 / 2 / 60%
ruff	–	4 / 4 / 0 / 100%
hadolint	–	2 / 2 / 0 / 100%
gitleaks	–	1 / 1 / 0 / 100%

`both` so với `llm`: precision +4,4 điểm, recall −5,5 điểm, F1 +1,0

- Thêm static làm LLM nói ít hơn hẳn: 57 finding thô so với 81 (prompt dặn không lặp lại lỗi static đã thấy), và phần LLM nói thêm bị gộp vào cụm static cùng dòng. Vì vậy số FP "thiếu test cho hàm mới" của QA giảm từ 13 xuống 6.
- **Hai case devops bị mất (`docker-latest`, `docker-apt-no-clean`) là lỗi nhãn, không phải lỗi phát hiện.** Ở cả hai case Claude (vai DevOps) báo đúng dòng, đúng category `devops`. Nhưng trên cùng dòng semgrep cũng báo (`avoid-latest-version`, `missing-assume-yes-switch`) và `static_tools.collect()` gán mọi luật semgrep không có chữ "security" trong metadata thành `bug`. Luật gộp "mọi finding cùng dòng là một cụm, finding static làm chính" khiến cụm mang nhãn `bug`, còn nhãn `devops` của Claude và hadolint chỉ còn trong phần đính kèm. Ground truth các case Dockerfile chấp nhận `devops` / `security` nên bị chấm là FN + FP. Gemini `both` cũng mất `docker-apt-no-clean` đúng theo cách này (xem `eval/out-v3-rescored/results.md`).
- **Đề xuất sửa** (chưa làm): trong `collect()`, luật semgrep đến từ bộ `dockerfile` (hoặc file là `Dockerfile`) gán `category="devops"`. Ước tính nếu sửa, `both` thành 35 TP / 11 FP / 1 FN → precision 76,1%, recall 97,2%, F1 ≈ 85,4%, không cần chạy lại LLM. Không thể rescore offline vì category được gán lúc đọc `semgrep.json` thô, file này không được lưu.

13 FP của `both` là gì

- 4 từ static: bandit B404/B403 (`import subprocess` / `import pickle`, low, 2 case); 2 nhãn semgrep ở trên; semgrep `dockerfile-source-not-pinned` dòng 1 case `docker-sudo` (dòng FROM nằm trong hunk).
- 9 từ LLM: 4 "thiếu test cho hàm mới" (QA); 5 nhận xét hợp lý ngoài ground truth: `yaml.load` thiếu `Loader` crash với PyYAML ≥ 6 (đúng), `hash_password` bỏ salt / hồi quy hash cũ (suy đoán), `parse_duration` cắt chữ số cuối (2 vai cùng báo, 2 dòng khác nhau), không escape CSV (đúng nhưng không phải lỗi được tiêm).

`static` so với lần đo 07/10

Cùng 21 TP, nhưng FP giảm 9 → 5: 4 finding semgrep `dockerfile-source-not-pinned` trên dòng `FROM` không thuộc hunk thay đổi nay bị bộ lọc static-theo-hunk (thêm sau lần đo v3, commit `2133adf`) loại bỏ. Thời gian 106,9 s/PR (v3: 101,4), 115 s cho case Python, ~68 s cho case chỉ có Dockerfile.

Chấm "đúng từng dòng" (`results_tol0.md`, lệch dòng = 0)

Config	Precision	Recall	F1
`static`	69,2%	50,0%	58,1%
`llm`	67,3%	97,2%	79,5% (không đổi)
`both`	65,2%	83,3%	73,2%

Claude ghim đúng dòng cho cả 35 TP (nhờ `existing_code`); số `both` tụt là do static tools báo lệch dòng (semgrep báo đầu khối, bandit báo dòng import).

2. Bộ đa ngôn ngữ (18 case JS/TS/Java/Go)

Config	TP	FP	FN	Precision	Recall	F1	s/PR (TB)	s/PR (trung vị)	tokens/PR
`llm`	16	9	2	64,0%	88,9%	74,4%	40,2	41,5	16 056
`both`	16	12	2	57,1%	88,9%	69,6%	115,1	117,9	14 475

Tham chiếu Gemini (`eval/out-multi-v3-rescored`): `llm` 75,0 / 100 / 85,7 · `both` 69,2 / 100 / 81,8.

Theo category (`llm` = `both`): bug 6/7, performance 0/1, security 7/7, test 3/3.

- **Hai case sót ở cả hai cấu hình đều là bất đồng nhãn, Claude đã nhìn thấy lỗi:**
- `js-concat-loop` (dòng 20, `performance`): Claude báo đúng dòng 20 nhưng về việc không escape CSV (`bug`), không nhắc nối chuỗi O(n^2). Gemini bắt được.
- `ts-missing-await` (dòng 16, `bug`): Claude báo đúng dòng, đúng vấn đề ("ép Promise thành Record do bỏ `await` trước `res.json()`") nhưng gắn `maintainability` / `low` nên không khớp danh sách category chấp nhận. Đây là lỗi runtime thật, nhãn Claude chọn sai.
- Static trên bộ này chỉ có semgrep (6 finding, 5 TP) và gitleaks (1 TP); không có hadolint/ruff/bandit. 5 TP đó LLM đều đã bắt, nên `both` không thêm recall, chỉ thêm ~75 s/PR và 3 FP (semgrep `no-direct-write-to-responsewriter` ở Go; hai vai cùng báo đổi thuật toán hash ở `java-md5`; QA báo `isAdmin` dùng `==` lỏng là `security` cạnh ground truth `bug`).
- FP của `llm` (9): 6 là QA "thiếu test cho hàm mới" / nhận xét test, 3 là SE nhận xét đúng ngoài ground truth. Mẫu giống hệt bộ Python và giống Gemini: vai QA là nguồn FP chính ở mọi model.

3. Vận hành

- Tổng 230 request tới Claude (gồm 2 request chạy thử), **0 lỗi, 0 timeout, 0 retry**; request lâu nhất 32 s. Chi phí theo giá niêm yết API ≈ 7,76 USD (thực tế chạy bằng quota tài khoản Claude Code).
- Bước fact-check **không loại bỏ finding nào** trong cả 4 lần chạy (`removed_by_factcheck = 0` ở mọi `_meta`). Với Claude, lớp kiểm chứng thứ hai hiện không có tác dụng lọc; cần so với số của Gemini (`eval/out-v3/*/_meta`) trước khi kết luận là do model hay do prompt fact-check.
- Một case `static` (`eq-none`) bị ghi timeout 28 834 s vì máy ngủ 8 giờ giữa lúc chạy (container đã quét xong, chỉ tiến trình ngoài bị tính giờ); đã chạy lại riêng (120,8 s, kết quả bình thường). Bài học: với lần chạy nhiều giờ, cần giữ máy thức (tiến trình nền gọi `SetThreadExecutionState`) hoặc chạy trên máy khác.
- Thời gian máy thực chạy ≈ 5 giờ: static 35 case ≈ 62 phút, both 35 case ≈ 103 phút, multi llm 18 case ≈ 12 phút, multi both 18 case ≈ 35 phút. Server Claude chạy 2 request song song, eval chạy tuần tự.

4. Kết luận

- Trên bộ Python, với Claude thì `both` là cấu hình tốt nhất (F1 80,5 so với 79,5 của `llm`), và sẽ là ≈ 85,4 nếu sửa nhãn semgrep cho luật Dockerfile. Với Gemini, `llm` vẫn hơn `both` (89,5 so với 87,2) vì Gemini vốn đã ít FP nên static chỉ thêm FP của chính nó.
- Claude recall cao hơn hoặc bằng Gemini trên cả hai bộ ở mức "có nhìn thấy lỗi" (bộ đa ngôn ngữ: 18/18 nếu tính cả 2 case lệch nhãn), nhưng precision theo ground truth thấp hơn 15–20 điểm vì viết nhiều nhận xét đúng ngoài ground truth và vì chọn nhãn category khác người soạn bộ test. Token/PR gấp ~2,2 lần Gemini (custom mode), thời gian/PR gấp ~2 lần.
- Hai điểm nên sửa trong pipeline, không phụ thuộc model: (1) nhãn semgrep cho luật Dockerfile → `devops`; (2) khi gộp finding LLM vào cụm static mà category khác nhau, nên giữ category LLM trong `categories` phụ để score và policy không mất thông tin. Điểm (1) ảnh hưởng trực tiếp 2 case ở Claude và 1 case ở Gemini.

Thư mục kết quả: `eval/out-claudecode-custom/` (`static`, `llm`, `both`, `results.md`, `results_tol0.md`), `eval/out-multi-claudecode/` (`llm`, `both`). Nhật ký từng case: `runs_<config>.json`.

5. Cập nhật 30/10/2026: đã sửa hai điểm ở mục 4, chấm lại offline

Hai sửa trong pipeline (không đụng model, không đụng prompt):

1. `static_tools.collect()`: luật semgrep về hạ tầng (nhận ra qua `metadata.technology`, tiền tố `check_id`, hoặc tên file Dockerfile / workflow CI) mang nhãn `devops` thay vì `bug`; luật semgrep xếp loại security vẫn là `security`.
2. `Finding.categories`: khi `postprocess.dedupe()` gộp một finding khác category vào cụm, nhãn đó được giữ ở trường phụ; `eval/score.py` chấm trúng nếu nhãn chính **hoặc** nhãn phụ nằm trong danh sách chấp nhận.

Chấm lại bằng `eval/rescore.py` (áp lại hậu xử lý mới lên finding đã lưu; với kết quả cũ không còn `semgrep.json` gốc nên nhãn semgrep được suy từ tên file + mã luật, tức hai trong ba tiêu chí của `collect()`). Điểm (2) chưa đo được offline vì finding cũ đã gộp rồi, không còn category của phần bị gộp; cần chạy lại LLM.

Bộ Python, 35 case	Precision	Recall	F1	Ghi chú
Claude `static` trước → sau	80,8 → 88,5	58,3 → 63,9	67,7 → **74,2**	2 FP thành TP
Claude `both` trước → sau	71,7 → 76,1	91,7 → 97,2	80,5 → **85,4**	đúng như dự đoán ở mục 4
Claude `llm`	67,3	97,2	79,5	không đổi (không có semgrep)
Gemini `both` trước → sau	81,0 → 83,3	94,4 → 97,2	87,2 → **89,7**	lần đầu `both` ≥ `llm` (89,5) với Gemini
Gemini `llm`	85,0	94,4	89,5	không đổi

Thư mục: `eval/out-claudecode-custom-relabel/`, `eval/out-v3-relabel/` (bị gitignore như mọi `eval/out-*`; tạo lại bằng `python eval/rescore.py --src <cũ> --dst <mới>` rồi `python eval/score.py --out <mới>`). Test: `tests/test_category_labels.py`.

6. Chạy lại thật 30/10 → 02/11/2026 với hai sửa ở mục 5 (Claude, cùng cấu hình mục 1)

Cùng server `claude-server/` chế độ `custom`, `claude-opus-5-5`, Docker bật cho static. Cấu hình `llm` không chạy lại vì không có semgrep và dedupe LLM–LLM chỉ gộp khi cùng nhãn, nên hai sửa không đụng tới nó. Lần

chạy bị ngắt hai lần vì máy ngủ / phiên đóng; `run_eval.py` chạy tiếp từ case dở, hai case có thời gian ảo (`docker-sudo` 29 giờ, `react-missing-key` 8,8 giờ) được chạy lại riêng bằng `--cases` nên cột thời gian là số thật.

Bộ · cấu hình	15/10 (nhãn cũ)	30/10 chấm lại offline	**02/11 chạy thật**	Ghi chú
Python · `static`	67,7	74,2	**76,2** (P 88,9 · R 66,7)	thêm 1 TP so với offline
Python · `both`	80,5	85,4	**86,4** (P 77,8 · R 97,2)	FP 13 → 10, FN 3 → 1; 149 s/PR, 15,3k token/PR
Python · `llm`	79,5	–	**79,1** (P 68,0 · R 94,4), chạy 10/11	TP 34, FP 16, FN 2; 43,5 s/PR, 17,1k token/PR — làm baseline cho cổng lớp 4 (mục 7.5)
Đa ngôn ngữ · `both`	69,6	–	**85,7** (P 75,0 · R 100)	TP 16 → 18, FP 12 → 6, FN 2 → 0; 118 s/PR
Đa ngôn ngữ · `llm`	74,4	–	không chạy lại	

Đọc số đa ngôn ngữ cho đúng: hai FN của 15/10 chính là "2 case lệch nhãn" đã ghi ở mục 2 (LLM thấy lỗi nhưng chọn category khác người soạn) — nay nhãn phụ `categories` cứu được, và một FP cũ đồng thời thành TP. Phần FP giảm thêm (12 → 6) không tách riêng được giữa sửa nhãn semgrep và dao động giữa hai lần gọi LLM (model không lặp lại y hệt); muốn tách phải chạy nhiều lần lấy trung bình. Với bộ Python, chạy thật cao hơn chấm lại offline 1 điểm ở cả hai cấu hình, cùng chiều, nên hai sửa coi như đã được xác nhận bằng lần chạy thật.

Fact-check (điểm mở ở mục 3) — so Claude với Gemini trên `_meta`: Claude `llm` 0/58 lần kiểm chứng loại finding, `both` 0/40; Gemini (`eval/out-v3`) `llm` 0/59, `both` 1/59. Tức là bước fact-check gần như không loại gì ở **cả hai model** trên bộ lỗi tiêm: nguyên nhân là bộ đề (mỗi lỗi có bằng chứng rõ ngay trong diff), không phải model hay prompt. Giá trị của bước này chỉ đo được trên PR thật (`eval/aacr/`), nơi LLM hay "nhìn thấy" lỗi không có trong diff.

Thư mục: `eval/out-claudecode-v2/` (`static`, `both`, từ 10/11 thêm `llm`), `eval/out-multi-claudecode-v2/` (`both`). CI: từ 02/11 bộ test chạy tự động trên GitHub Actions (`.github/workflows/test.yml`), lần đầu xanh.

16 FP của Claude `llm` (10/11) là gì: `eval/judge_fp.py` với giám khảo **khác họ** (Gemini 3.1 flash-lite, 3 phiếu) kết luận **16/16 đúng về code, 0 sai sự thật**: 5 là "thiếu test cho hàm/endpoint mới", còn lại là lỗi có thật ngoài lỗi được tiêm (subprocess không timeout, endpoint luôn trả 200, CMD dạng shell không nhận SIGTERM, `sudo` trong image slim…). Precision theo nhãn 68,0% là cận dưới; tính cả nhận xét đúng ngoài nhãn là 100%. Con số này quyết định cách làm mục 7.5.

7. PR thật (AACR-Bench, 15 PR Python) với Claude — 02/11/2026

Cùng bộ 15 PR và 68 vấn đề chuẩn như `eval/aacr/out/` (Gemini). Model review và giám khảo đều là `claude-opus-5-5` (3 phiếu/vấn đề), nên con số này **chưa** có giám khảo khác họ (việc D14). Chạy từ 01:53 đến 03:51 (~2 giờ cho 2 cấu hình).

Model	Config	Finding	TP chặt	P	R	F1	TP lỏng	F1 lỏng
Gemini 3.5 flash-lite	llm	8	1	12,5%	1,5%	2,6%	1	2,6%
Gemini 3.5 flash-lite	both	286	2	0,7%	2,9%	1,1%	2	1,1%
Claude Opus 5.5	llm	42	14	33,3%	20,6%	**25,5%**	17	30,9%
Claude Opus 5.5	both	62	15	24,2%	22,1%	23,1%	18	27,7%

Theo mức ngữ cảnh (Claude `llm`, chặt): Diff Level 1/21, File Level 8/29, Repo Level 5/18. Với `both`: 1/21, 9/29, 5/18.

Nhận xét: - Khoảng cách model trên PR thật lớn hơn nhiều so với bộ lỗi tiêm (mục 1: Claude và Gemini chỉ cách nhau vài điểm). Trên PR thật Gemini gần như không bắt được gì (1 đến 2 TP), Claude bắt 14 đến 15. Bộ lỗi tiêm vì thế không đủ để phân biệt model; PR thật mới lộ ra. - Static tools gần như không giúp trên PR thật: `both` thêm 20 finding nhưng chỉ thêm 1 TP, F1 giảm 2,4 điểm so với `llm`. Ngược với bộ lỗi tiêm (nơi `both` cao hơn `static` / `llm` rõ). Lý do: vấn đề chuẩn của AACR chủ yếu là lỗi logic cần ngữ cảnh file/repo (47/68), loại semgrep/bandit/ruff không bắt. - Điểm yếu còn lại: Diff Level chỉ bắt 1/21 ở cả hai cấu hình, dù đây là nhóm "chỉ cần nhìn diff". Cần xem từng case trong `eval/aacr/out-claudecode/llm/` và `both/` để biết là bỏ sót hay giám khảo không khớp mô tả. - Giám khảo cùng model với bên review có thể thiên vị; trước khi đưa vào báo cáo chính cần chấm lại với giám khảo khác họ (D14), hoặc ít nhất lấy mẫu 10 TP đọc tay.

Thư mục: `eval/aacr/out-claudecode/` (gitignore; `results.md`, `runs_llm.json`, `runs_both.json`, thư mục `llm/`, `both/` chứa phán quyết giám khảo). Lệnh: `LLM_PROFILE=claudecode python eval/aacr_eval.py --steps run,score --configs llm,both --out out-claudecode`.

7.1 Giám khảo khác họ (D14) — 02/11/2026

Chấm lại nguyên các finding đã sinh, chỉ đổi giám khảo (`eval/aacr_eval.py --steps score --judge-tag <tên> --review-model <model>`; cache và results mang hậu tố riêng). Mỗi chiều 3 phiếu/vấn đề.

Finding của	Giám khảo	Config	TP chặt	P	R	F1	TP lỏng
Claude Opus 5.5	Claude (tự chấm, mục 7)	llm	14	33,3%	20,6%	25,5%	17
Claude Opus 5.5	**Gemini 3.1 flash-lite**	llm	15	35,7%	22,1%	27,3%	16
Claude Opus 5.5	Claude (tự chấm)	both	15	24,2%	22,1%	23,1%	18
Claude Opus 5.5	**Gemini 3.1 flash-lite**	both	16	25,8%	23,5%	24,6%	18
Gemini flash-lite	Gemini (tự chấm, `eval/aacr/out`)	llm	1	12,5%	1,5%	2,6%	1
Gemini flash-lite	**Claude Opus 5.5**	llm	1	12,5%	1,5%	2,6%	1
Gemini flash-lite	Gemini (tự chấm)	both	2	0,7%	2,9%	1,1%	2
Gemini flash-lite	**Claude Opus 5.5**	both	3	1,0%	4,4%	1,7%	3

Kết luận: đổi giám khảo làm số TP lệch tối đa 1 ở mọi ô, cùng chiều cho cả hai model. Không có dấu hiệu giám khảo cùng họ "thiên vị" bên sinh: Gemini chấm Claude còn cao hơn Claude tự chấm 1 TP, Claude chấm Gemini cao hơn Gemini tự chấm 1 TP. Vì vậy khoảng cách Claude ≈ 25 F1 so với Gemini ≈ 2 F1 trên PR thật là thật, không phải do cách chấm. Ghi chú vận hành: Gemini trả 503 quá tải nhiều lần trong ngày; scorer đã thêm gọi lại (tối đa 6 lần, chờ tăng dần) và không ghi cache khi thiếu quá nửa số phiếu (commit `0eedc6a`), nếu không một phiếu lỗi sẽ bị tính thành "không khớp" vĩnh viễn trong cache.

Kết quả: `eval/aacr/out-claudecode/results_judge_gemini.md`, `eval/aacr/out/results_judge_claude.md`.

7.2 Đọc tay 21 vấn đề Diff Level (D15)

Nhóm Diff Level là nhóm "chỉ cần nhìn diff" nhưng Claude chỉ bắt 1/21 (2/21 theo Gemini chấm). Đọc từng vấn đề chuẩn (`eval/aacr/dataset.json`, tập con 15 PR) và các finding cùng file của cấu hình `llm`:

Loại vấn đề chuẩn	Số	Bot bắt	Ví dụ
Văn phong / tài liệu / đặt tên (không phải lỗi)	14	0	lỗi chính tả trong comment, hai dấu cách trước `noqa`, gợi ý walrus `:=`, `not x == 0` → `x != 0`, thụt lề 8 thay vì 4, dùng `cls` thay tên class, đặt tên hằng cho số 16384
Lỗi logic thật	7	1	**bắt:** xoá cạnh tuần tự thay vì song song (ragflow 6691). **sót:** dấu phẩy thừa trong `.format(i,)` (ComfyUI 6033); hai chỗ `d["key"]` thiếu `.get` (ragflow 6691, langflow 5388); chuỗi filter bắt đầu bằng `" AND"` khi rỗng (ragflow 7151); `@lru_cache` nhận cả `*args` làm hỏng "log once" và tăng bộ nhớ (vllm 17425, 2 vấn đề)

14 vấn đề nhóm đầu HermesQA **cố ý không đăng** (ngưỡng `min_severity_to_comment=medium`, checklist loại góp ý văn phong), nên recall thấp ở Diff Level phần lớn là lựa chọn thiết kế, không phải bỏ sót. 6 lỗi logic sót mới là điểm yếu thật: đều là lỗi "nhìn kỹ một dòng" (dấu phẩy, `.get`, chuỗi rỗng, cache key) trong khi finding Claude đưa ra ở cùng file lại là lỗi luồng xử lý lớn hơn. Hướng cải thiện: thêm mục "đọc từng dòng đổi: đối số thừa, truy cập dict không `.get`, chuỗi nối với điều kiện rỗng, cache key" vào checklist `python.md`, rồi đo lại bằng cổng eval ở lớp 4.

7.3 Lớp 4 chạy thật lần đầu (C9) — 02/11/2026

Sau thử nghiệm phản hồi trên PR #1 (`docs/THU_NGHIEM_PHAN_HOI_2026-11-02.md`), bảng `feedback` có 6 dòng thật. Chạy `tools/learn_rules.py propose --docs python.md --profile claudecode`: Claude nhận 5 bằng chứng (4 reject, 1 accept, đều trên `examples/c8_demo*.py`) và **không đề xuất dòng nào** cho checklist. Đây là kết quả đúng: bằng chứng chỉ nói "thư mục examples/ là demo cố ý", không rút được luật chung cho mọi repo; đề xuất luật từ 5 phản hồi trên một thư mục demo sẽ là học thuộc đề. Bước gate (C10) vì thế chưa có ứng viên để chạy; cần phản hồi từ PR thật đa dạng hơn.

7.4 A/B memory bật/tắt trên chuỗi PR cùng repo (C11) — 10/11/2026

Cách đo. Worker nạp repo memory từ Postgres, còn CLI/eval trước đây không nạp gì, nên không đo được lớp 2–3 ngoài bot. Bổ sung `app.cli --memory-file` (ghi chú từ file JSON, chọn theo lô file + vai bằng đúng hàm `memory.rank_memory` của worker) và `--reflect-out` (sau review, agent tự rút ghi chú rồi nối vào file). `eval/aacr_eval.py --memory-chain --repos infiniflow_ragflow` chạy 4 PR ragflow của AACR theo thứ tự số PR tăng dần (5065 → 6553 → 6691 → 7151): PR sau nhận ghi chú agent rút từ PR trước. Đối chứng: cùng 4 PR, cùng model (`claude-opus-5-5`, `llm`), không memory, chạy lại cùng ngày (`out-claudecode-nomem`); thêm lần chạy gốc 02/11 (`out-claudecode`) để thấy dao động giữa hai lần chạy giống hệt nhau. Chấm bằng cả hai giám khảo (Gemini và Claude, 3 phiếu) và phân loại finding ngoài nhãn bằng Gemini (`eval/aacr_fp_judge.py`).

Arm (4 PR, 27 vấn đề chuẩn)	Finding	TP chặt · Gemini chấm	TP chặt · Claude chấm	P / R / F1 (Gemini)	Ngoài nhãn: đúng / sai	Token vào	Thời gian
Gốc 02/11, không memory	17	6	5	35,3 / 22,2 / 27,3	–	194k	608 s
Chạy lại 10/11, không memory	17	8	6	47,1 / 29,6 / 36,4	9 / 0	196k	489 s
Có memory (chuỗi)	22	**9** (lỏng 10)	**8**	40,9 / 33,3 / 36,7	12 / 0	209k	619 s

Ghi chú agent tự rút trong chuỗi (file `eval/aacr/out-claudecode-mem/memory/infiniflow_ragflow.json`), số ghi chú mỗi PR nhận được là 0 / 1 / 2 / 2:

1. (sau 5065) `web/src/` dùng i18n cho chuỗi hiển thị; chuỗi UI hard-code là lệch quy ước — *convention*.

2. (sau 6553) `api/db/services/` là dữ liệu đa tenant: truy vấn theo tên/id cần lọc thêm kb_id/tenant_id — *pattern*.
3. (sau 7151) `rag/utils/infinity_conn.py` là một trong các backend doc store; sửa một backend nên soát các backend còn lại.
4. (sau 7151) `infinity_conn.py` ghép filter thành chuỗi kiểu SQL, giá trị đưa vào cần escape dấu nháy đơn.

Đọc kết quả. Có memory nhiều hơn 1–2 TP so với đối chứng cùng ngày, nhưng hai lần chạy *không memory* đã lệch nhau 2 TP (6 so với 8), nên chênh lệch này nằm trong dao động của model; precision theo nhãn giảm vì có thêm 5 finding, song giám khảo khác họ không thấy finding nào sai ở cả hai arm. Quan trọng hơn: cả 4 ghi chú đều nói về thư mục mà PR *kế tiếp* không chạm tới (5065 sửa `web/src`, 6553 sửa `api/`, 6691 sửa `graphrag/`, 7151 sửa `rag/utils` + `graphrag/`), nên thí nghiệm này thực chất đo "ghi chú không liên quan có làm review xấu đi không" (không, theo số liệu) chứ chưa đo được lợi ích của memory. Muốn thấy lợi ích cần chuỗi PR dài hơn trong cùng một vùng code, hoặc ghi chú từ người (lớp 1) — chính là điều PR #1 (mục 7.3, `docs/THU_NGHIEM_PHAN_HOI_2026-11-02.md`) đã cho thấy trên repo demo. Cơ chế thì đã chạy trọn vòng ngoài DB: ghi chú đúng vùng code, đúng kiểu (convention/pattern), không có ghi chú tắt bảo mật. Chi phí memory: +6,7% token vào, +1 lần gọi LLM (reflect) mỗi PR.

7.5 Cổng eval của lớp 4 chạy thật (C10) — 10/11/2026

Không có bằng chứng "bác bỏ" để đề xuất luật. Thử ba nguồn: (a) 6 phản hồi thật trên PR #1 (mục 7.3); (b) 24 finding của Claude không khớp nhãn trên 15 PR thật AACR, cho Gemini chấm (`eval/aacr_fp_judge.py`): **23 đúng về code, 1 sai** (precision theo nhãn 42,9% → 97,6% nếu tính nhận xét đúng ngoài nhãn); (c) 16 FP của Claude `llm` trên bộ lỗi tiêm (mục 6): 16/16 đúng. Gộp (a)+(b) rồi (a)+(b)+(c nửa A) thành 6 và 12 bằng chứng cho `propose --evidence-file`: Claude **không đề xuất dòng nào** cả hai lần — đúng nguyên tắc "một loại nhận xét bị bác bỏ ≥ 2 lần", vì chỉ có đúng 1 nhận xét bị kết luận sai. Nói cách khác, trên dữ liệu hiện có, "báo động giả" của HermesQA hầu hết là nhận xét đúng nhưng ngoài phạm vi nhãn, không phải thứ lớp 4 được thiết kế để học.

Chạy cổng với ứng viên viết tay. Để kiểm tra cổng bằng eval thật, viết tay một dòng "Không báo" kiểu mà người bảo trì dễ bị cám dỗ thêm, dựa trên mẫu lặp lại nhiều nhất trong 16 FP (5 lần "thiếu test"): `Thiếu unit test cho hàm hoặc endpoint mới khi PR không chạm thư mục tests/`. Tách đôi bộ lỗi tiêm xen kẽ theo thứ tự case (nửa A 18 case lấy bằng chứng, nửa B 17 case chấm cổng, cân bằng theo loại lỗi) để không học thuộc đề; `gate --cases <nửa B>` chấm baseline mục 6 trên cùng 17 case (bổ sung tham số `cases` cho `learn_rules.metrics`).

```
python tools/learn_rules.py gate --doc python.md --baseline eval/out-claudecode-v2 --out eval/out-learned \
    --config llm --profile claudecode --cases <17 case nửa B>
KHÔNG ĐẠT  precision 61.5% -> 58.3% (-3.2%), recall 94.1% -> 82.4% (-11.8%), F1 74.4% -> 68.3%
giữ nguyên checklist; ứng viên vẫn ở data/rules_candidates/python.md để xem tay
```

Overlay làm đúng điều nó hứa: finding loại `test` trong nửa B giảm 4 → 1, nhưng trong đó có TP duy nhất của case `new-func-no-test` (lỗi tiêm chính là "thiếu test") → thành FN; `string-concat-loop` và `cmd-os-system` bớt FP "thiếu test". Case `docker-apt-no-clean` mất TP là dao động của model (Dockerfile không dùng `python.md`). Kết luận: **cổng từ chối đúng** một luật giảm nhiễu nhưng làm mất recall; thư mục `app/skills/rules/learned/` vẫn trống, checklist không đổi. Chi phí một lần gate trên 17 case: ~13 phút Claude. Điểm yếu thật của lớp 4 lộ ra không phải ở cổng mà ở nguồn bằng chứng: sau 4 lần thử chưa có mẫu "bị bác bỏ lặp lại" nào ngoài thư mục demo; cần người dùng thật 👇 trên nhiều repo.